PHILIPS MR TEAM · ADVANCED AGENTIC AI PROGRAM · 4–5 NOVEMBER 2026

Advanced Agentic AI: Spec, Build, Guard, Break

Write the spec, direct Codex to build a multi-agent app with MCP, then set guardrails, evaluate and red-team it — on ChatGPT Enterprise and Azure-hosted models

CLIENT
Philips MR Team

DATES
4–5 Nov 2026 · 2 days

PLATFORM
ChatGPT Enterprise + Azure AI

FORMAT
In-person · browser · no coding needed

SPEC
Spec → Plan → Tasks

BUILD
Agents + MCP + UI

GUARD
Content Safety + eval harness

BREAK
Red-team & harden

unext

WHAT THE TEAM LEAVES WITH

Four Artefacts Every Team Takes Home

Specified by the team, built by Codex, reviewed by the team — reusable the next working day.

Executable spec

Written by the team: spec.md that Codex turns into plan.md and tasks.md — intent, scope, data, tools, guardrails, acceptance tests.

Working agentic app

Built by Codex from the spec: FastAPI on the 10-point scaffold, LangGraph agents, an MCP agent, the team's data and a Streamlit UI.

Guarded + evaluated

Azure AI Content Safety plus four app-level guard layers; rules and pass thresholds set by the team, checked by an eval harness.

Red-team report

Attacks typed into the app, a log of every break, the fix for each, and a resilience scorecard against agreed thresholds.

2 days, in person

9 agentic use cases, one per team

20 OWASP risks mapped to the app

~70% hands-on lab time

Browser, ChatGPT Enterprise, Azure — How It Fits

Participants work in the browser and the Codex app; the agents call AI models hosted in Philips' Azure environment.

Participant laptop

- Chrome or Edge
- ChatGPT Enterprise: Chat, Work, Codex
- Codex app (signed in)
- Spec + AGENTS.md templates

Agentic app (built by Codex)

- Streamlit UI
- FastAPI service (10-point scaffold)
- LangGraph orchestrator + agents
- MCP server + use-case data

Azure — Philips

- Claude Sonnet / Haiku 5.5
- GPT-5.5 · GPT 5.6 Luna · GPT-4.1
- Azure AI Content Safety
- Entra ID or key access

Ground rules: Philips ChatGPT Enterprise only · sanitised or synthetic data only — no PHI, patient, customer or personal data · red-team only the workshop app.

Five Azure-Hosted Models, Each With a Job

Pick the cheapest model that passes the eval — the agent design decides, not habit.

Claude Sonnet 5.5	Main reasoning / orchestrator agent	*Anthropic Messages API*
Claude Haiku 5.5	Fast classification and summaries	*Anthropic Messages API*
GPT-5.5	Alternate orchestrator; side-by-side comparison	*Azure OpenAI API*
GPT 5.6 Luna	Role confirmed after the trainer's test	*Azure OpenAI API*
GPT-4.1	Fallback and low-cost baseline	*Azure OpenAI API*

Two API styles, one port

- Claude on Azure uses Anthropic's Messages API at a /anthropic endpoint
- GPT models use the Azure OpenAI API
- The app hides both behind one model port — switching model is a config change
- The eval harness runs the same golden set on each model

Philips provides deployment names, endpoints, Entra ID or key access, and quota for ~30 people working at once.

Day 1 Builds It; Day 2 Guards It and Breaks It

DAY 1 · Wed 4 Nov — Spec-Driven Build

09:00–09:30	Kickoff + environment check
09:30–11:00	S1 · Spec-driven development: Spec ⟶ Plan ⟶ Tasks ⟶ Code
11:15–13:00	S2 · Lab: write the spec in ChatGPT; Codex drafts plan + tasks
14:00–14:45	S3 · Orchestration, MCP, LangChain · LangGraph · CrewAI — live demos
14:45–15:45	S4a · Build with Codex: agents, MCP server, MCP agent
16:00–17:15	S4b · Build with Codex: Skills, data, Streamlit UI, review + run
17:15–17:30	Demo round · teams share output file

DAY 2 · Thu 5 Nov — Guard, Evaluate, Break

09:00–09:30	Day 1 recap · why guardrails
09:30–11:00	S5 · OWASP LLM Top 10 + Agentic Top 10, mapped to our app
11:15–13:00	S6 · Lab: Content Safety, guard layers, eval rules + thresholds
14:00–15:30	S7 · Red-team: type attacks into another team's app
15:45–16:45	S8 · Fix, re-run the harness, resilience scorecard
16:45–17:30	Showcase, takeaways and close

Breaks: 11:00–11:15 tea · 13:00–14:00 lunch · afternoon tea 15:45 (Day 1) / 15:30 (Day 2). Teams of 3: spec owner · Codex driver · reviewer.

DAY 1 · WEDNESDAY 4 NOVEMBER

Spec-Driven Build

Environment check and spec-driven development in the morning. Orchestration, MCP and Skills demos, then direct Codex to build your team's agent app in the afternoon.

unext

Six Green Ticks Before We Start

The same five-minute self-check from the prerequisites sheet. The trainer has already validated the Azure side.

Browser

Chrome or Edge on the room Wi-Fi or VPN; the workshop app link opens.

ChatGPT sign-in

chatgpt.com shows the Philips workspace, not a personal account.

Chat

Ask “Summarise what an AI agent is in 3 bullets” — a normal reply.

Codex

Open Codex and send “Say hello” — it responds with no access error.

Codex app

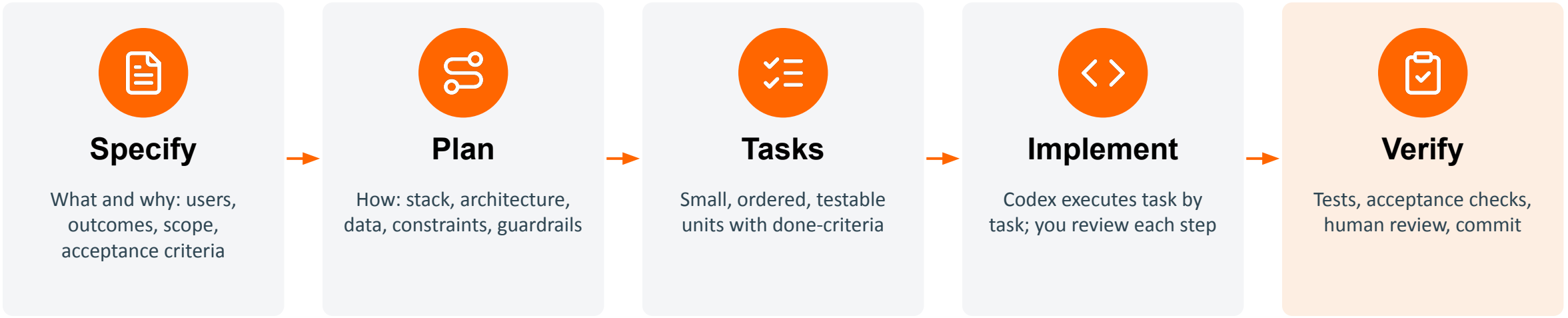
The Codex app is installed on the laptop and signed in.

Work

The Work option is visible in the ChatGPT account.

Spec-Driven Development: The Spec Is the Truth

Prompts are disposable; specs are versioned. The coding agent executes the spec — it does not guess intent.



Prompt-driven (vibe coding)

- Intent lives in chat history and is lost
- Output drifts between runs and developers
- Review happens after the code exists

Spec-driven

- Intent is a versioned file the agent reads every run
- Regenerate or extend from the same spec
- Review the spec and plan before code is written

Anatomy of a Spec an Agent Can Execute

Each team writes this in ChatGPT for its use case, then has Codex generate plan.md and tasks.md from it.

1	**Intent and users**	Who uses it, what job it does, what success looks like
2	**Scope and non-goals**	What the agent must never do — refusals belong here
3	**Data and classification**	Entities, sources, PUBLIC / INTERNAL / CONFIDENTIAL tags
4	**Agents and MCP tools**	Roles, handoffs, tool list with argument schemas
5	**Guardrails**	Input, tool-call, output and approval rules, declared up front
6	**Acceptance tests**	Given / when / then cases the eval harness will run
7	**Non-functionals**	Latency, cost ceiling, audit log, the 10-point scaffold

Workshop project folder

```
specs/002-defect-hotspot/
   spec.md · plan.md · tasks.md
AGENTS.md              ← rules for Codex
data/                  ← sanitised sample
skills/                ← SKILL.md packs
app/  core/  adapters/  mcp_server/
tests/  evals/  redteam/
```

Nine Agentic Use Cases — One Team Each

Each team of three owns one use case. Agent 9 combines the other eight into one view of team and programme health.

1 Scrum Master Agent

Delivery plan from priorities, dependencies and capacity; sprint health; stand-up summaries

2 Defect Trend & Hotspot

Component × severity, hotspot modules and files, inflow vs outflow, MTTR

3 AI Regulations

Track regulation changes, assess impact on QMS procedures, flag gaps for review

4 Field Issues Summary

Group Trackwise and ClearQuest field issues, find recurring patterns, top must-fix

5 Pull Request Flow

Build time and failures, PR cycle time, review wait, bottlenecks

6 CI/CD Pipeline Health

Build failures, flaky tests, consolidation frequency, recurring failure patterns

7 Top MEBEF Issues

Top 10 issues lowering MEBEF per release, by subsystem, existing CQ check

8 SBOM Vulnerability

Look up vulnerabilities for SBOM components, rank by severity, track to resolution

9 Dashboard Orchestrator

Combines agents 1–8 into one view of health, trends, risks and attention areas

Five Orchestration Patterns and When to Use Them

Sequential

Fixed pipeline; each step feeds the next. Predictable, cheap, easy to test.

Supervisor

Orchestrator routes work to specialist agents and merges results. Our Day 1 build.

Router / handoff

Classify once, hand control to the right agent with typed state.

Parallel fan-out

Independent sub-tasks run concurrently, then aggregate.

Evaluator loop

Generator plus critic iterate until a rubric passes or a limit is hit.

Failure modes we design against

Infinite loops

Fix: recursion limit + step budget

Context bloat

Fix: typed state, not raw transcripts

Cascading errors

Fix: validate every handoff

Lost work

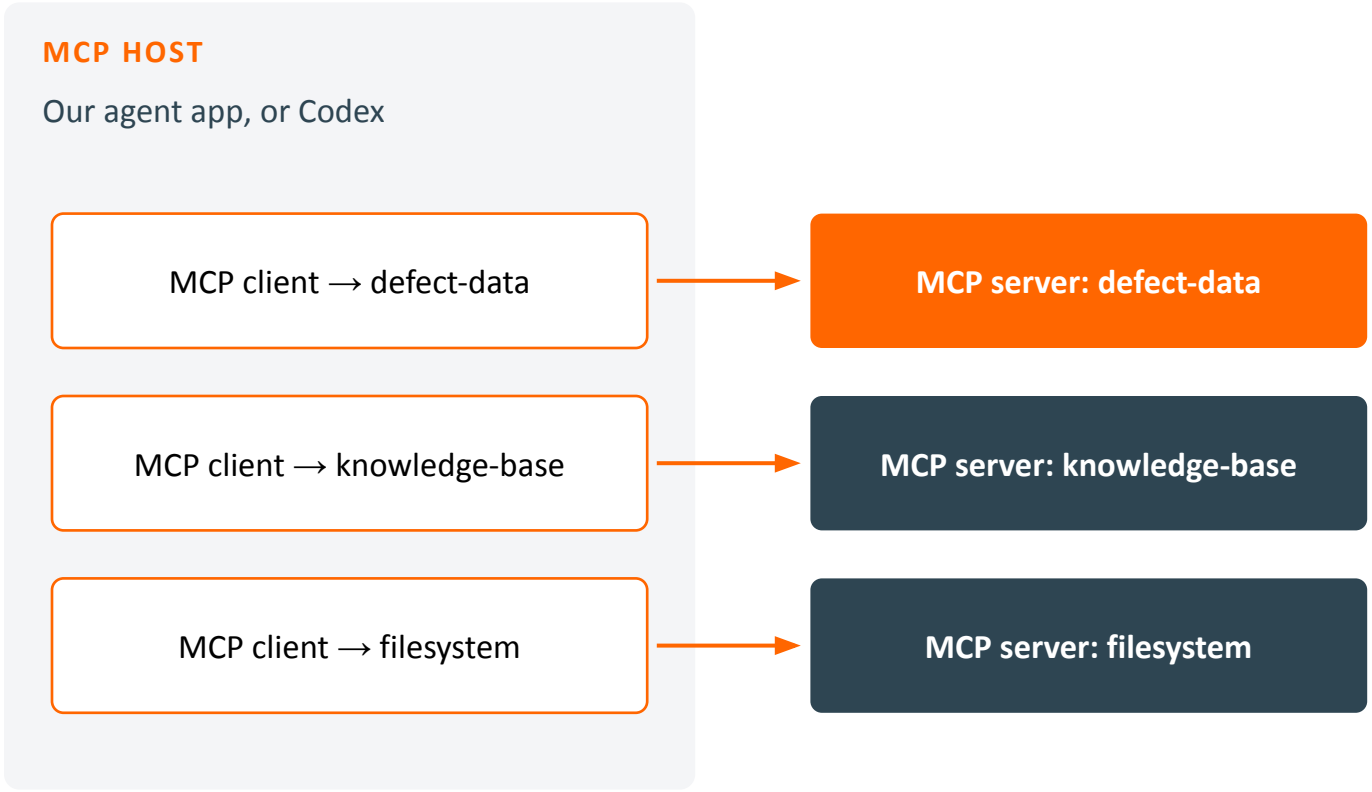
Fix: checkpoints and resumable runs

LangChain, LangGraph, CrewAI — Which Does What

	LangChain	LangGraph	CrewAI
Mental model	Building blocks: models, tools, prompts, retrievers, create_agent	Stateful graph: nodes, edges, typed shared state	Role-based crew: agents, tasks, sequential or hierarchical process
Best at	Single agent with tool calling; model and tool wrappers	Production orchestration with loops, branches and retries	Fast prototyping of a team of specialist roles
State and HITL	Middleware and memory; basic	Checkpointing, interrupts for human approval, time-travel	Crew memory; human input on tasks; Flows for control
Azure models	Azure OpenAI and Anthropic chat-model wrappers	Same model objects as LangChain	LLM configured with an azure/ or anthropic model id
In this workshop	Azure model + tool wrappers	**Main orchestrator for the Day 1 app**	Stretch: rebuild the same flow and compare

Rule of thumb: start with LangGraph when you need control and auditability; use CrewAI to explore role designs quickly; LangChain supplies the parts for both.

MCP: Tools Agents Can Discover and Call



stdio locally · Streamable HTTP when remote

Primitives

Tools (actions), Resources (read-only context), Prompts (templates)

What we build

A server per use case over the team's data (e.g. defect-data) with typed, read-only tools — used by our MCP agent and by Codex

Security

Least-privilege tools, argument schemas, auth on remote servers, allowlisted servers only — tool descriptions are untrusted input

The 10-Point FastAPI Scaffold Every Team Builds On

Codex applies these through a Skill; you review every change against them. The harness is what lets a coding agent change code safely.

1. **Type hints on every boundary**
   Wrong types rejected where they enter
2. **Pydantic v2 strict models**
   All LLM output parsed, never raw dicts
3. **Async I/O throughout**
   No blocking calls inside the event loop
4. **Typed client, retries + backoff**
   Timeout, jitter, capped attempts, fallback
5. **Provider behind a port**
   Azure GPT and Claude are adapters, swappable
6. **Tool args validated first**
   Schema check before any tool code runs
7. **Guards on destructive tools**
   Business rules, authorisation, human approval
8. **Request and response models**
   Bad data kept out, secrets kept in
9. **Contract tests as a CI gate**
   Assert shape and properties, not wording
10. **Pure core**
    No SDK, SQL or framework imports in the domain

How the Day 1 App Is Layered

Streamlit UI — Results view · agent trace · approve / reject

↓

FastAPI airlock — Request + response models · auth · /triage /approve /health

↓

LangGraph orchestrator — Supervisor ⟶ triage, knowledge, resolution agents · checkpoints · interrupt for human gate

↓

Pure core — Use cases, domain models, ports — no SDK, SQL or framework imports

↓

LLM port	**Tool port**	**Repo port**
Azure OpenAI + Anthropic adapters	MCP client ⟶ use-case server	Team data (CSV / SQLite)

Guard slots, empty on Day 1

- L1 input pre-hook — before the orchestrator
- L2 tool-call guard — inside the tool port
- L3 output validator — before the response model
- L4 audit + human gate — at the interrupt

Day 2 fills each slot — no re-architecture needed.

Skills Teach the Coding Agent How Your Team Builds

A skill is a folder with a SKILL.md and optional scripts and references, loaded on demand by the coding agent.

Skills vs MCP — not the same thing

Skills
Shape how the coding agent writes code: conventions, checklists, templates, scripts. Used at build time.

MCP tools
Actions the runtime agent calls while serving a user: search defects, read an SBOM. Used at run time.

Codex loads Skills automatically when a task matches their description.

```
---
name: fastapi-ai-scaffolding
description: Use when building a FastAPI
  service that calls an LLM ...
---
```

`fastapi-ai-scaffolding`
Builds and audits the 10-point scaffold — Day 1

`mcp-tool-builder`
Typed FastMCP tools with schemas and guards — Day 1

`guardrail-hooks`
Wires Azure AI Content Safety and L1–L4 hooks — Day 2

`redteam-runner`
Runs attack suites and writes the break log — Day 2

Six Checkpoints From Spec to Running App



Scaffold

Generated by Codex from the spec; checks green

Model adapter

Azure GPT + Claude, retries, strict parsing

Agents

LangGraph supervisor + 3 agents

MCP

Use-case MCP server + MCP agent wired

Data + UI

Team data loaded; Streamlit front end

Demo

End-to-end run; output file shared

How the lab runs

- Codex executes tasks.md one task at a time
- Reviewer checks each result before the next task
- Trainer gate at every checkpoint; ready-built fallback per CP

Stretch goals for fast teams

- Rebuild the flow in CrewAI and compare traces
- Run the golden set on two Azure models
- Add a fourth agent from a new spec section

DAY 2 · THURSDAY 5 NOVEMBER

Guard, Evaluate, Break

Why guardrails and eval harnesses matter, OWASP LLM and Agentic Top 10s, Azure AI Content Safety in the app — then red-team the Day 1 build and harden it.

unext

Why Guardrails and an Eval Harness Are Not Optional

Why guardrails

- Agents act: a bad output becomes a bad database write
- Every defect, log or field-issue text is untrusted input
- Regulated MR context: data boundaries must be provable
- Model-side safety alone does not know your business rules

Why an eval harness

- Model output varies run to run; unit tests alone flap
- Model, prompt or guardrail changes silently shift quality
- A fixed golden set turns “seems fine” into a number
- As a CI gate it blocks regressions before they ship

Guardrails stop known-bad behaviour at runtime. The harness proves, on every change, that they still do — and that the app still does its job.

OWASP Top 10 for LLM Applications (2025)

LLM01 — **Prompt Injection**
Instructions hidden in a defect description

LLM02 — **Sensitive Info Disclosure**
Site or engineer details leak in answers

LLM03 — **Supply Chain**
Unpinned packages, third-party MCP servers

LLM04 — **Data and Model Poisoning**
Poisoned SOPs in the knowledge base

LLM05 — **Improper Output Handling**
Model output rendered or executed unchecked

LLM06 — **Excessive Agency**
Agent posts or updates without approval

LLM07 — **System Prompt Leakage**
Prompt reveals rules, keys or endpoints

LLM08 — **Vector and Embedding Weaknesses**
Retrieval crosses data classifications

LLM09 — **Misinformation**
Confident fix with no cited source

LLM10 — **Unbounded Consumption**
Agent loops burn tokens and cost

OWASP Top 10 for Agentic Applications (2026)

ASI01 — **Agent Goal Hijack**
Field-issue text redirects the supervisor's goal

ASI02 — **Tool Misuse and Exploitation**
Data tool called with no date range or limit

ASI03 — **Identity and Privilege Abuse**
Agent uses an over-privileged Azure key

ASI04 — **Agentic Supply Chain**
Unvetted MCP server or tool description

ASI05 — **Unexpected Code Execution**
Generated code or shell run without a sandbox

ASI06 — **Memory and Context Poisoning**
Bad case written to memory skews later runs

ASI07 — **Insecure Inter-Agent Comms**
Untyped handoffs between our three agents

ASI08 — **Cascading Failures**
Triage error propagates to resolution and DB

ASI09 — **Human-Agent Trust Exploitation**
Polished draft rubber-stamped at the human gate

ASI10 — **Rogue Agents**
Agent drifts from its spec over many runs

Defence in Depth: Azure AI Content Safety + App Layers

Azure AI Content Safety

Prompt Shields — user-prompt and document (indirect) attacks

Harm categories — hate, sexual, violence, self-harm by severity

Blocklists and custom categories — e.g. clinical diagnosis advice

PII detection — detect and redact personal data

Groundedness detection — answers tied to retrieved sources

Protected material detection — known text and code

Called as a service, so it screens text for every model — Claude and GPT deployments alike.

L1

Input pre-hook

Prompt Shields on user text and on record text (descriptions, logs, notes) before the orchestrator

L2

Tool-call guard

Schema, allowlist, business rule, authorisation and call budget on every MCP call

L3

Output validator

Harm + PII check on output, strict model parse, citation required

L4

Audit + human gate

Structured log per step; human approval for any write or notify action

An Eval Harness That Gates Every Change

Golden set

30–50 cases from the spec's acceptance tests: normal, edge and adversarial

Graders

Deterministic checks first (schema, enum, citations); LLM-as-judge with a rubric for the rest

CI gate

pytest + promptfoo run on every PR; merge blocked below thresholds

Metric	How measured	Gate
Task success	Rubric score on golden set	**≥ 90%**
Tool-call accuracy	Right tool, valid args	**≥ 95%**
Groundedness	Citations match sources	**≥ 90%**
Injection block rate	Attack cases stopped	**≥ 95%**
PII leakage	Scanner on all outputs	**0**
p95 latency · cost	Per request, from logs	**Agreed SLO**

Thresholds are proposals; teams agree final values in the spec.

Red-Team the Day 1 App: Teams Swap and Attack

Each team attacks another team's app by typing into its chat screen; the trainer runs scripted suites. Every break is logged.

Direct prompt injection

LLM01 · ASI01

Indirect injection via record text

LLM01 · ASI01

Tool misuse: empty or broad filter

LLM06 · ASI02

PII and secret exfiltration

LLM02 · LLM07

MCP tool-description poisoning

ASI04

Knowledge-base poisoning

LLM04 · ASI06

Rubber-stamp the human gate

ASI09

Loop and cost exhaustion

LLM10 · ASI08

Tooling

promptfoo red-team

Plugin-based attack generation against the API

garak

LLM vulnerability probes

PyRIT

Multi-turn, scripted attack orchestration

redteam/ pytest suite

Trainer-run scripts on MCP tools and handoffs

Harden, Re-Test, Score — Then Show It Live



Showcase: each team in 5 minutes

- Spec in → agents running on Azure models
- The attack that worked against them
- The guardrail layer that now stops it, with the audit log
- Scorecard before and after hardening

Resilience scorecard	Target
Breaks fixed and re-tested	100%
Breaks added as regression cases	100%
Unauthorised tool calls	0
PII / secret leakage	0
Eval harness pass rate	≥ 90%
Audit log completeness	100%

What Must Be in Place Before 09:00, 4 November

No coding background needed and nothing to install beyond the Codex app — everything else runs in the browser.

Participants bring

- Laptop with Chrome or Edge, and charger
- ChatGPT Enterprise sign-in: Chat, Work, Codex
- Codex app installed and signed in
- Problem statement for the use case
- Sanitised data sample + 10–15 test questions with answers
- One subject-matter expert in the team

Philips provides

- Five Azure model deployments: names, endpoints, access, quota
- Trainer access today, incl. one Claude test call
- Azure AI Content Safety
- Codex and Work enabled by the ChatGPT admin
- Network access to Azure AI endpoints
- Data owner approval for the samples

UNext provides

- Ready-built demo apps connected to the Azure models
- Spec and AGENTS.md templates
- Sample data and Skills
- Evaluation set and red-team attack scripts
- Trainer on site both days

Thank You

unext